CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY & RESEARCH

Department of Computer Science & Engineering

Subject Name: Java Programming

Semester:3
Subject Code: CSE201
Academic year: 2024-2025

Part - 1

No.	Aim of the Practical

2	Imagine you are developing a simple banking application where you need to display the current balance of a user account. For simplicity, let's say the current balance is $20.Write a java program to store this balance in a variable and then display it to the user.
	PROGRAM CODE :
	import java.util.Scanner;
	
	public class Bank {
	public static void main(String[] args) {
	Scanner sc = new Scanner(System.in);
	System.out.print("Enter The Bank Balence :");
	int a = sc.nextInt();
	System.out.print("your balance is : "+a +" Doller"):
	}

```
}
```

OUTPUT:

```
C:\Users\HP>cd "c:\Users\HP\Desktop\Prectical code\java\" && javac Bank.java && java Bank
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter The Bank Balence :20
your balance is : 20  Doller
c:\Users\HP\Desktop\Prectical code\java>
```

CONCLUSION:

In this programme We understood how we can take input from user and how we print the output..

2. Write a program to take the user for a distance (in meters) and the time taken (as three numbers: hours, minutes, seconds), and display the speed, in meters per second, kilometers per hour and miles per hour (hint:1 mile = 1609 meters).

PROGRAM CODE :

```
import java.util.Scanner;


public class Speed {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
       System.out.print("Enetr the totel distance in meater :");
       float a= sc.nextFloat();
       System.out.print("Enetr the totel time in formet hh:mm:ss :");
       float h=sc.nextFloat();
       float m=sc.nextFloat();
       float s=sc.nextFloat();
       float time_s=(h*60*60)+(m*60)+s;
       float time_m=(h*60)+m+(s/60);
       float time_h=h+(m/60)+(s/3600);


       System.out.println("Which formet you want speed ");
```

```
        System.out.println("1 - for meater per second");
        System.out.println("2 - for km per hour");
        System.out.println("3 - for mile per hour");
        int res=sc.nextInt();
        if (res==1){
        System.out.printf("your speed in m/s is :"+ a/time_s);
        }
        if(res==2){
        System.out.printf("your speed in km/hour is :"+ (a/1000)/time_h );
}
if (res==3){
        System.out.printf("your speed in mile/hour is :"+(a/1609)/time_h);
}
        }
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java>cd "c:\Users\HP\Desktop\Prectical code\java\"
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enetr the totel distance in meater :8000
Enetr the totel time in formet hh:mm:ss :0
57
27
Which formet you want speed
1 - for meater per second
2 - for km per hour
3 - for mile per hour
1
your speed in m/s is :2.3208587
```

CONCLUSION:

We've created a program that takes the distance and time as input, calculates the speed in different units, and displays the results

4. Imagine you are developing a budget tracking application. You need to calculate the total expenses for the month. Users will input their daily expenses, and the program should compute the sum of these expenses. Write a Java program to calculate the sum of elements in an array representing daily expenses.

PROGRAM CODE :

```
import java.util.Scanner;

public class Expance {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        float expance[] =new float[30];
        float sum=0;


        for(int i=1;i<30;i++){
        System.out.print("Enter the expance of  day " +i +" : ");
        expance[i]= sc.nextFloat();
        sum=sum+expance[i];


        }
        System.out.println("your totel expance is : " +sum);
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java>cd "c:\Users\HP\Desktop\Prectical code\java\" && javac Expance.java && java Expance
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the expance of  day 1 : 200
Enter the expance of  day 2 : 300
Enter the expance of  day 3 : 100
Enter the expance of  day 4 : 50
Enter the expance of  day 5 : 28
Enter the expance of  day 6 : 800
Enter the expance of  day 7 : 700
Enter the expance of  day 8 : 900
Enter the expance of  day 9 : 10000
Enter the expance of  day 10 : 5
Enter the expance of  day 11 : 20
Enter the expance of  day 12 : 30
Enter the expance of  day 13 : 85
Enter the expance of  day 14 : 456
Enter the expance of  day 15 : 85
Enter the expance of  day 16 : 74
Enter the expance of  day 17 : 17
Enter the expance of  day 18 : 98
Enter the expance of  day 19 : 85
Enter the expance of  day 20 : 52
Enter the expance of  day 21 : 78
Enter the expance of  day 22 : 25
Enter the expance of  day 23 : 763
Enter the expance of  day 24 : 200
Enter the expance of  day 25 : 850
Enter the expance of  day 26 : 85
Enter the expance of  day 27 : 123
Enter the expance of  day 28 : 521
Enter the expance of  day 29 : 456
your totel expance is : 17186.0
```

CONCLUSION:

By this experiment, I learnt about how we can apply loops in java and how we can perform any operation on various type of variable in java.

5. An electric appliance shop assigns code 1 to motor,2 to fan,3 to tube and 4 for wires. All other items have code 5 or more. While selling the goods, a sales tax of 8% to motor,12% to fan,5% to tube light,7.5% to wires and 3% for all other items is charged. A list containing the product code and price in two different arrays. Write a java program using switch statement to prepare the bill.

PROGRAM CODE :

```
import java.util.*;

public class PT_5 {
    public static void main(String arg[]){
        Scanner s = new Scanner(System.in);
        int price[] = {1000,650,300,100,50};
        double tax[] = {0.08,0.12,0.05,0.075,0.03};
        int bill = 0;
        System.out.println("1 - Motor");
        System.out.println("2 - Fan");
        System.out.println("3 - Tube");
        System.out.println("4 - Wires");
        System.out.println("5 - Other Items");

        System.err.print("Enter number of Items : ");
        int num = s.nextInt();
        int list[] = new int[num];

        for(int i=0 ; i<num ; i++){
```

```
        System.out.print("Enter Item Number : ");

            list[i] = s.nextInt();
        }

        for(int i=0 ; i<num ; i++){
            switch(list[i]){
                case 1 : bill += price[0] + price[0]*tax[0];
                break;
                case 2 : bill += price[1] + price[1]*tax[1];
                break;
                case 3 : bill += price[2] + price[2]*tax[2];
                break;
                case 4 : bill += price[3] + price[3]*tax[3];
                break;
                case 5 : bill += price[4] + price[4]*tax[4];
                break;
                default : System.out.println("Enter Valid Product Code");
                break;
            }
            System.out.println("Product Code : " + list[i] + " Price : " + price[list[i]-1] + "
Tax : " + tax[list[i]-1]);
        }
        System.out.println("Your Total Bill : " + bill);
    }
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java>cd "c:\Users\HP\Desktop\Prectical code\java\java
PT_5
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
1 - Motor
2 - Fan
3 - Tube
4 - Wires
5 - Other Items
Enter number of Items : 1
Enter Item Number : 2
Product Code : 2 Price : 650 Tax : 0.12
Your Total Bill : 728

c:\Users\HP\Desktop\Prectical code\java\java>
```

CONCLUSION:

By this experiment, I learnt about do-while loop, switch statement and array. We can easily make a good system with this feature of java language.also I realize how we can apply array in real life problem.

6. Create a Java program that prompts the user to enter the number of days (n) for which they want to generate their exercise routine. The program should then calculate and display the first n terms of the Fibonacci series, representing the exercise duration for each day.

PROGRAM CODE :

```
import java.util.Scanner;
 public class prectical6 {
 public static void main(String[] args) {
System.out.println("Fibonacci Series ");
Scanner sc = new Scanner(System.in);
System.out.print("Enter the number for size of series : ");
int n = sc.nextInt();
int n1 = 0, n2 = 1, n3;
System.out.print("0 1 ");
for (int i = 2; i < n; i++) {
n3 = n1 + n2;
n1 = n2;
n2 = n3;
System.out.print(n2+ " ");
          }
       }
   }
```

OUTPUT:

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Fibonacci Series
Enter the number for size of series : 10
0 1 1 2 3 5 8 13 21 34
c:\Users\HP\Desktop\Prectical code\java\java>
```

CONCLUSION:

By this practical, I learn about function and mthod. How we can make function or method of a class and how we can access in main function and also learnt recursive function in java programming.

PART-II Strings

7. Given a string and a non-negative int n, we'll say that the front of the string is the first 3 chars, or whatever is there if the string is less than length 3. Return n copies of the front;

front_times('Chocolate', 2) → 'ChoCho'

front_times('Chocolate', 3) → 'ChoChoCho'

front_times('Abc', 3) → 'AbcAbcAbc'

PROGRAM CODE :

```
import java.util.Scanner;
public class Prectical7{
public static void main (String[] args) {
Scanner sc = new Scanner(System.in);

System.out.print("Enter the size of the string: ");
int n = sc.nextInt();
String input ;
String response ;

System.out.print("Enetr the string : ");
input = sc.next();
System.out.println();

response =input.substring(0,3);

System.out.print("Enter the size you want to print: ");
int m = sc.nextInt();
for (int i = 0; i < m; i++) {
```

```
System.out.print( response);
}
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java>cd "c:\Users\HP\Desktop\Prectical
ava Preseven
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the size of the string: 6
Enetr the string : prince

Enter the size you want to print: 4
pripripripri
c:\Users\HP\Desktop\Prectical code\java\java>
```

CONCLUSION:

To solve the problem of generating n copies of the front part of a string in Java, use the substring function to extract the first three characters (or the entire string if it's shorter). Then, repeat this segment n times. This method ensures correct handling even if the string is shorter than three characters. The solution is efficient, using simple string slicing and repetition.

8. Given an array of ints, return the number of 9's in the array. array_count9([1, 2, 9]) →1
array_count9([1, 9, 9]) → 2
array_count9([1, 9, 9, 3, 9]) → 3

PROGRAM CODE :

```
import java.util.Scanner;

public class Prectical8{
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enetr the totel number of the Element in array :");
int n = sc.nextInt();
int[] arr = new int[n];
int count =0;
for(int i = 0; i < n; i++){
System.out.print("Enetr the "+(i+1) + " Element : ");
arr[i] = sc.nextInt();

}
for(int i=0;i<n;i++){
if(arr[i]==9){
count++;
}
}
System.out.print("Your array is :");
for(int i=0;i<n;i++){
System.out.print(arr[i]+" ");
```

```
}
System.out.println();
System.out.println("The totel nine in the array is :"+count);
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java>cd "c:\Users\HP\Desktop\Prectical
vac Prectical8.java && java Prectical8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enetr the totel number of the Element in array :5
Enetr the 1 Element : 4
Enetr the 2 Element : 9
Enetr the 3 Element : 8
Enetr the 4 Element : 9
Enetr the 5 Element : 6
Your array is :4 9 8 9 6
The totel nine in the array is :2
```

CONCLUSION:

To count the number of 9's in an integer array in Java, iterate through the array and use a counter to track occurrences of the number 9. This method ensures that each element is checked and counted accurately. The solution is straightforward, utilizing basic iteration and conditional checks, making it efficient and easy to implement. It effectively solves the problem by providing the exact count of 9's in the array, as demonstrated in various test cases.

9. Given a string, return a string where for every char in the original, there are two chars.

double_char('The') → 'TThhee'

double_char('AAbb') → 'AAAAbbbb'

double_char('Hi-There') → 'HHii--TThheerree'

PROGRAM CODE :

```
import java.util.Scanner;
public class Practic9 {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter a string: ");
String input = sc.nextLine();
int size = input.length();

for (int i = 0; i < size; i++) {
char ch = input.charAt(i);
System.out.print(ch + "" + ch);
}

sc.close();
}}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java>cd "c:\Users\HP\Desktop\Prectical code\java\java\"
ava Practic9
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter a string: Prince
PPrriinnccee
c:\Users\HP\Desktop\Prectical code\java\java>
```

CONCLUSION:

To double every character in a string in Java, iterate through each character, appending it twice to a new string. This method ensures each character is duplicated, creating a new string with repeated characters. It provides an efficient and straightforward solution for string manipulation tasks.

10. Perform following functionalities of the string:

- Find Length of the String
- Lowercase of the String
- Uppercase of the String
- Reverse String
- Sort the string

PROGRAM CODE :

```
import java.util.*;
public class Prectical10 {
public static void main(String[] args) {
Scanner sc=new Scanner(System.in);
String input;
System.out.print("Enter the string: ");
input=sc.nextLine();
System.out.println("Enetr 1 for the find lenghth of the String ");
System.out.println("Enetr 2 for convert string to the lower case  ");
System.out.println("Enetr 3 for convert string to the upper case  ");
System.out.println("Enetr 4 for revers the string: ");
System.out.println("Enetr 5 for the sort the string ");
int choice=sc.nextInt();
switch(choice){
case 1:
System.out.println("The length of the string is: "+input.length());
break;
case 2:
System.out.println("The lower case of the string is: "+input.toLowerCase());
```

```
break;
case 3:
System.out.println("The upper case of the string is: "+input.toUpperCase());
break;

case 4:
StringBuilder reversed = new StringBuilder(input);
reversed.reverse();
System.out.println("Reversed string: " + reversed.toString());
break;
case 5:
char[] charArray = input.toCharArray();
Arrays.sort(charArray);
String sortedString = new String(charArray);
System.out.println("Sorted string: " + sortedString);
break;
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java>cd "c:\Users\HP\Desktop\Prectical
vac Prectical10.java && java Prectical10
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the string: Prince
Enetr 1 for the find lenghth of the String
Enetr 2 for convert string to the lower case
Enetr 3 for convert string to the upper case
Enetr 4 for revers the string:
Enetr 5 for the sort the string
1
The length of the string is: 6
```

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the string: PRINCE
Enetr 1 for the find lenghth of the String
Enetr 2 for convert string to the lower case
Enetr 3 for convert string to the upper case
Enetr 4 for revers the string:
Enetr 5 for the sort the string
2
The lower case of the string is: prince
```

```
C:\Users\HP\Desktop\Prectical code>cd "c:\Users\HP\Desktop\Prectical code\java\
rectical10.java && java Prectical10
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the string: prince
Enetr 1 for the find lenghth of the String
Enetr 2 for convert string to the lower case
Enetr 3 for convert string to the upper case
Enetr 4 for revers the string:
Enetr 5 for the sort the string
3
The upper case of the string is: PRINCE
```

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the string: PRINCE
Enetr 1 for the find lenghth of the String
Enetr 2 for convert string to the lower case
Enetr 3 for convert string to the upper case
Enetr 4 for revers the string:
Enetr 5 for the sort the string
4
Reversed string: ECNIRP

c:\Users\HP\Desktop\Prectical code\java\java>
```

```
c:\Users\HP\Desktop\Prectical code\java\java>cd "c:\Users\HP\Desktop\Prectical code\java\java\"
&& javac Prectical10.java && java Prectical10
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the string: PRINCE
Enetr 1 for the find lenghth of the String
Enetr 2 for convert string to the lower case
Enetr 3 for convert string to the upper case
Enetr 4 for revers the string:
Enetr 5 for the sort the string
5
Sorted string: CEINPR
```

CONCLUSION:

To manipulate a string in Java, use built-in methods: length() to find the string's length, toLowerCase() and toUpperCase() for case conversions, and StringBuilder's reverse() for reversing the string. Sorting can be done by converting the string to a char array, using Arrays.sort(), and converting it back to a string. These methods provide efficient string handling.

11. Perform following Functionalities of the string:

"CHARUSAT UNIVERSITY"

- Find length
- Replace 'H' by 'FIRST LATTER OF YOUR NAME'
- Convert all character in lowercase

PROGRAM CODE :

```
import java.util.*;
public class prectical11 {
public static void main(String[] args) {
Scanner sc=new Scanner (System.in);
String input;

System.out.print("Enetr the input string  ");
input=sc.nextLine();

System.out.println("Enetr 1 for find length of the String ");
System.out.println("Enetr 2 for replace by h");
System.out.println("Enetr 3 for convert string to the lowercase : ");
System.out.println("Enetr your choice ");
int choice=sc.nextInt();
switch(choice){
case 1:
System.out.println("Length of the string is "+input.length());
break;
case 2:
System.out.println("Enter the character to be replaced ");
```

```
char ch=sc.next().charAt(0);
System.out.println("Enter the character to be replaced by");
char ch1=sc.next().charAt(0);
String s=input.replace(ch,ch1);
System.out.println("String after replacement is "+s);
break;
case 3:
String s1=input.toLowerCase();
System.out.println("String after conversion to lowercase is "+s1);
break;
default:
System.out.println("Invalid choice");
}
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java>cd "c:\Users\HP\Desktop\Prectical code\
vac prectical11.java && java prectical11
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enetr the input string  GHEVARIYA PRINCE
Enetr 1 for find length of the String
Enetr 2 for replace by h
Enetr 3 for convert string to the lowercase :
Enetr your choice
1
Length of the string is 16
```

```
c:\Users\HP\Desktop\Prectical code\java\java>cd "c:\Users\HP\Desktop\Prectical code\
vac prectical11.java && java prectical11
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enetr the input string  GHEVARIYA PRINCE
Enetr 1 for find length of the String
Enetr 2 for replace by h
Enetr 3 for convert string to the lowercase :
Enetr your choice
2
Enter the character to be replaced
G
Enter the character to be replaced by
H
String after replacement is HHEVARIYA PRINCE
```

```
c:\Users\HP\Desktop\Prectical code\java\java>cd "c:\Users\HP\Desktop\Prectical code\java\java\
vac prectical11.java && java prectical11
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enetr the input string  GHEVARIYA PRINCE
Enetr 1 for find length of the String
Enetr 2 for replace by h
Enetr 3 for convert string to the lowercase :
Enetr your choice
3
String after conversion to lowercase is ghevariya prince
```

CONCLUSION:

By this practical we learn various function of the string manipulation like change the character or replace the character in the string. For find length we use .length() function and for replace the character we first read the checter by using charAt() function and to convert upper case to lower case string we use toLover() function.

12

PART-III Object Oriented Programming: Classes, Methods, Constructors

Imagine you are developing a currency conversion tool for a travel agency. This tool should be able to convert an amount in Pounds to Rupees. For simplicity, we assume the conversion rate is fixed: 1 Pound = 100 Rupees. The tool should be able to take input both from command-line arguments and interactively from the user.

PROGRAM CODE :

```
import java.util.Scanner;
public class pre {
public static void main(String[] args) {
int pound;
if (args.length > 0) {
pound = Integer.parseInt(args[0]);
} else {
Scanner sc = new Scanner(System.in);
System.out.print("Enter amount in Pounds: ");
pound = sc.nextInt();
sc.close();
}
int ruppes = pound * 100;
System.out.println(pound + " Pounds is equal to " + ruppes + " Rupees.");
}
}
```

OUTPUT:

```
C:\Users\HP\Desktop\Prectical code\java>java pre.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter amount in Pounds: 20
20 Pounds is equal to 2000 Rupees.
```

```
C:\Users\HP\Desktop\Prectical code\java>java pre.java 20
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
20 Pounds is equal to 2000 Rupees.
```

CONCLUSION:

By this practical, I learnt about Classes and object. How we can make class and object in java programming.we can easily make constructor which initialize the value of data member. We also make member function of any class. By this practical, I also learnt that how we take the argument by command line argument.

13 Create a class called Employee that includes three pieces of information as instance variables—a first name (type String), a last name (type String) and a monthly salary (double). Your class should have a constructor that initializes the three instance variables. Provide a set and a get method for each instance variable. If the monthly salary is not positive, set it to 0.0. Write a test application named EmployeeTest that demonstrates class Employee's capabilities. Create two Employee objects and display each object's yearly salary. Then give each Employee a 10% raise and display each Employee's yearly salary

PROGRAM CODE :

```
import java.util.*;
class Employee {
Scanner sc =new Scanner (System.in);
String first_name= "";
String  last_name;
double salary;
Employee(){
}

Employee(String f,String l,double s){
first_name=f;
last_name=l;
salary=s;

}

void set_first_name() {
first_name = sc.nextLine();
```

```
}

void set_last_name() {
last_name = sc.nextLine();
}

void set_salary() {
salary = sc.nextDouble();
}

String get_first_name() {

return first_name;
}

String get_last_name() {
return last_name;
}

double get_salary(){
return salary;
}

}

public class EmpoleeyTest {
public static void main(String arr[]){
```

```
Employee s=new Employee();
System.out.print("Enetr the first name :");
s.set_first_name();
System.out.print("Enetr the last name :");
s.set_last_name();
System.out.print("Enetr the salary:");
s.set_salary();
System.out.print("Your first name is : ");
String m=s.get_first_name();
System.out.print(m);
System.out.println();
System.out.print("Your last name is : ");
String l=s.get_last_name();
System.out.print(l);
System.out.println();
double sa=s.get_salary();
if(sa<0){
sa=0;
}else{
sa=(sa*0.1) + sa;
}
System.out.print("Your updated salary is ");
System.out.print(sa);
}
}
```

OUTPUT:

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enetr the first name :Ghevariya
Enetr the last name :Prince
Enetr the salary:150000
Your first name is : Ghevariya
Your last name is : Prince
Your updated salary is 165000.0
c:\Users\HP\Desktop\Prectical code\java>
```

CONCLUSION:

By this practical, I learnt about Classes and object. How we can make class and object in java programming.we can easily make constructor and member function of any class. By this practical, I also learnt how we deals with object and how we can apply any type of operation on data variable or objects.

14 Create a class called Date that includes three pieces of information as instance variables—a month (type int), a day (type int) and a year (type int). Your class should have a constructor that initializes the three instance variables and assumes that the values provided are correct. Provide a set and a get method for each instance variable. Provide a method displayDate that displays the month, day and year separated by forward slashes (/). Write a test application named DateTest that demonstrates class Date's capabilities.

PROGRAM CODE :

```
import java.util.Scanner;
 class Date {
Scanner sc = new Scanner(System.in);
int mon;
int day;
int year;
Date(){
 }
Date(int m,int d,int y){
mon=m;
day=d;
year=y;
}
void set_mon()
{
mon=sc.nextInt();
}
void set_year() {
year = sc.nextInt();
```

```
}

void set_day() {
day = sc.nextInt();
}

int get_mon() {
return mon;
}

int get_day() {
return day;
}

int get_year() {
return year;
}

void get_c(){
System.out.print(day +"/" + mon +"/" +year +" ");
}
}

public class DateTest {
public static void main(String arg[]){
Date s =new Date();
```

```
System.out.print("Enetr the day :");
s.set_day();
System.out.print("Enetr the month  :");
s.set_mon();
System.out.print("enter the year :");
s.set_year();
s.get_c();
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java>cd "c:\Users\HP\Desktop\Prectical code\java\"
st.java && java DateTest
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enetr the day :25
Enetr the month  :8
enter the year :2024
25/8/2024
c:\Users\HP\Desktop\Prectical code\java>
```

CONCLUSION:

By this practical, I learnt about how we get the data variable or class members and how we will set the value of data variable or object by member function and how we will change the value of data variable and members with the use of member function.

15 Write a program to print the area of a rectangle by creating a class named 'Area' taking the values of its length and breadth as parameters of its constructor and having a method named 'returnArea' which returns the area of the rectangle. Length and breadth of rectangle are entered through keyboard.

PROGRAM CODE :

```
import java.util.*;

class area {
int l;
int b;
area(){

}

public area(int len, int bre) {
l = len;
b = bre;
}

public int returnarea() {
return l * b;
}
}

public class return_area {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter the length of the rectangle: ");
int len = sc.nextInt();
System.out.print("Enter the breadth of the rectangle: ");
int bre = sc.nextInt();

area rectangle = new area(len, bre);

System.out.println("The area of the rectangle is: " + rectangle.returnarea());

sc.close();
}
}
```

OUTPUT:

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the length of the rectangle: 50
Enter the breadth of the rectangle: 12
The area of the rectangle is: 600
```

CONCLUSION:

By this practical , I learnt about how we will return any value through member function and how we will perform any mathematical operation using class and object.

16. Print the sum, difference and product of two complex numbers by creating a class named 'Complex' with separate methods for each operation whose real and imaginary parts are entered by user.

PROGRAM CODE :

```
import java.util.*;
public class practical16
{
  public static void main(String args[])
  {
    Scanner in = new Scanner(System.in);
    System.out.println("ENTER FIRST COMPLEX NUMBER : ");
    float x1=in.nextFloat();
    float y1=in.nextFloat();
    complex c1=new complex(x1, y1);
    System.out.println("ENTER SECOND COMPLEX NUMBER : ");
    float x2=in.nextFloat();
    float y2=in.nextFloat();
    complex c2=new complex(x2, y2);
    System.out.println("FIRST NUMNER : ");
    c1.print();
    System.out.println("SECOND NUMNER : ");
    c2.print();
    complex c3=new complex();
    complex c4=new complex();
    complex c5=new complex();
    c3=c1.sum(c2);
```

```
        c4=c1.diff(c2);
        c5=c1.mux(c2);
        System.out.println("SUM : ");
        c3.print();
        System.out.println("DIFFERENCE : ");
        c4.print();
        System.out.println("MULTIPLICATION : ");
        c5.print();
    }
}
class complex{
    private float x;
    private float y;
    complex(){}
    complex(float x, float y)
    {
        this.x=x;
        this.y=y;
    }
    complex sum(complex c)
    {
        complex temp=new complex();
        temp.x=x+c.x;
        temp.y=y+c.y;
        return temp;
    }
    complex diff(complex c)
```

```
  {
    complex temp=new complex();
    temp.x=x-c.x;
    temp.y=y-c.y;
    return temp;
  }
  complex mux(complex c)
  {
    complex temp=new complex();
    temp.x=x*(c.x)-y*(c.y);
    temp.y=x*(c.y)+y*(c.x);
    return temp;
  }
  void print()
  {
    if(y>0)
    {
    System.out.println(x+"+i"+y);}
    else{
      System.out.println(x+"-i"+(-y));}
  }
}
```

OUTPUT :

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
ENTER FIRST COMPLEX NUMBER :
16
20
ENTER SECOND COMPLEX NUMBER :
78
25
FIRST NUMNER :
16.0+i20.0
SECOND NUMNER :
78.0+i25.0
SUM :
94.0+i45.0
DIFFERENCE :
-62.0-i5.0
MULTIPLICATION :
748.0+i1960.0

c:\Users\HP\Desktop\Prectical code\java>
```

CONCLUSION :

By this practical, I learnt about how we will perform mathematical operation using classes and object. I also learnt from this practical that how we make member function of class datatype and how we return the value.

Part -4

No.	Aim of the Practical
17.	Create a class with a method that prints “thparent class" and its subclass with another method that prints "This is child class". Now, create an object for each of the class and call 1 - method of parent class by object of parent

PROGRAM CODE:

```
class Parent{
   void parent(){
      System.out.println("This is  Parent class");
   }

}

class Child extends Parent{
   void child(){
      System.out.println("This is Child class");
   }
}

public class Practical17{
   public static void main(String arg[]){
   Parent p = new Parent();
   Child c = new Child();
   p.parent();
   c.child();
   }
```

```
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_4>cd "c:\Users\HP\Desktop\Prec
ical code\java\java\prectical_List\SET_4\" && javac Practical17.java && java Practical17
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
This is  Parent class
This is Child class
```

CONCLUSION:

In object-oriented programming, a parent class can have its methods invoked using its own object. The child class, through inheritance, can access both its own methods and those of the parent class using the child object. Each class object operates independently of the other.

18. Create a class named 'Member' having the following members: Data members

1 - Name

2 - Age

3 - Phone number

4 - Address

5 – Salary

It also has a method named 'printSalary' which prints the salary of the members. Two classes 'Employee' and 'Manager' inherits the 'Member' class. The 'Employee' and 'Manager' classes have data members 'specialization' and 'department' respectively. Now, assign name, age, phone number, address and salary to an employee and a manager by making an object of both of these classes and print the same.

PROGRAM CODE:

```
import java.util.*;

class member {
   String name;
   int age;
   long pho;
   String add;
   int sal;

   member() {
   }

   public static int printSalary(int sal) {
      return sal;
   }
```

```
}

class manager extends member {
   String dept;
}

class employee extends member {
   String spec;
}

public class practical18 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);



      employee emp = new employee();
      System.out.println("Enter employee details:");
      System.out.print("Name: ");
      emp.name = sc.nextLine();
      System.out.print("Age:  ");
      emp.age = sc.nextInt();
      System.out.print("Phone number: ");
      emp.pho = sc.nextLong();
      sc.nextLine();
      System.out.print("Address: ");
      emp.add = sc.nextLine();
      System.out.print("Salary:  ");
      emp.sal = sc.nextInt();
```

```
        sc.nextLine();
        System.out.print("Specialization: ");
        emp.spec = sc.nextLine();



        manager mgr = new manager();
        System.out.println("Enter manager details:");
        System.out.print("Name: ");
        mgr.name = sc.nextLine();
        System.out.print("Age:  ");
        mgr.age = sc.nextInt();
        System.out.print("Phone number: ");
        mgr.pho = sc.nextLong();
        sc.nextLine();
        System.out.print("Address: ");
        mgr.add = sc.nextLine();
        System.out.print("Salary: ");
        mgr.sal = sc.nextInt();
        sc.nextLine();
        System.out.print("Department: ");
        mgr.dept = sc.nextLine();


       int sam= member.printSalary(emp.sal);
       System.out.println("The salary of the employee is :"+sam);


       int sam1= member.printSalary(mgr.sal);
       System.out.println("The salry of the manager is :"+sam1);
```

```
    }
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_4>cd "c:\Use
ical code\java\java\prectical_List\SET_4\" && javac practical18.java && java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter employee details:
Name: Utsav
Age: 22
Phone number: 1234567890
Address: surat
Salary: 500000
Specialization: devops
Enter manager details:
Name: Prince
Age: 25
Phone number: 9925128371
Address: surat
Salary: 200000
Department: cse
The salary of the employee is :500000
The salry of the manager is :200000
```

CONCLUSION:

Both the Employee and Manager classes inherit common attributes from the Member class, allowing them to share common functionality such as printing salary. This demonstrates code reusability and simplified maintenance via inheritance.

19. Create a class named 'Rectangle' with two data members 'length' and 'breadth' and two methods to print the area and perimeter of the rectangle respectively. Its constructor having parameters for length and breadth is used to initialize length and breadth of the rectangle. Let class 'Square' inherit the 'Rectangle' class with its constructor having a parameter for its side (suppose s) calling the constructor of its parent class as 'super(s,s)'. Print the area and perimeter of a rectangle and a square. Also use array of objects.

PROGRAM CODE

```
import java.util.*;

class rectancle {
   int len;
   int brea;

   rectancle(int len, int brea) {
      this.len = len;
      this.brea = brea;
   }

   void area() {
      int area = len * brea;
      System.out.println("Area: " + area);
   }

   void perimeter() {
      int perimeter = 2 * (len + brea);
      System.out.println("Perimeter: " + perimeter);
   }
}

class square extends rectancle {
   square(int side) {
```

```
        super(side, side);
    }
}

public class Practical19 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        System.out.println("Enter number of rectancles: ");
        int n = in.nextInt();
        rectancle[] rec= new rectangle[n];

        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter length and breath of rectancle " + (i + 1) + ": ");
            int len = in.nextInt();
            int brea = in.nextInt();
            rec[i] = new rec(len, brea);
        }

        System.out.println("\nArea and Perimeter of rectancles: ");
        for (int i = 0; i < n; i++) {
            System.out.println("rectancle " + (i + 1) + ":");
            rec[i].area();
            rec[i].perimeter();
        }

        System.out.println("\nEnter number of squares: ");
        int m = in.nextInt();
        square[] squ = new squre[m];

        for (int i = 0; i < m; i++) {
```

```
            System.out.println("\nEnter side of square " + (i + 1) + ": ");
            int side = in.nextInt();
            squ[i] = new squ(side);
        }

        System.out.println("\nArea and Perimeter of Squares: ");
        for (int i = 0; i < m; i++) {
            System.out.println("Square " + (i + 1) + ":");
            squ[i].area();
            squ[i].perimeter();
        }
    }
}
```

OUTPUT:

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter number of rectancles:
2

Enter length and breath of rectancle 1:
10
15

Enter length and breath of rectancle 2:
30
2

Area and Perimeter of rectancles:
rectancle 1:
Area: 150
Perimeter: 50
rectancle 2:
Area: 60
Perimeter: 64

Enter number of squares:
2

Enter side of square 1:
10

Enter side of square 2:
30
```

```
Area and Perimeter of Squares:
Square 1:
Area: 100
Perimeter: 40
Square 2:
Area: 900
Perimeter: 120

c:\Users\HP\Desktop\Prectical code\java\java\prectic
```

CONCLUSION:

By inheriting the Rectangle class, the Square class reuses the logic for calculating area and perimeter with minimal changes. Using an array of objects allows for handling multiple instances efficiently, showcasing inheritance, polymorphism, and code reusability in object-oriented programming.

20

Create a class named 'Shape' with a method to print "This is This is shape". Then create two other classes named 'Rectangle', 'Circle' inheriting the Shape class, both having a method to print "This is rectangular shape" and "This is circular shape" respectively. Create a subclass 'Square' of 'Rectangle' having a method to print "Square is a rectangle". Now call the method of 'Shape' and 'Rectangle' class by the object of 'Square' class.

PROGRAM CODE:

```
class Shape {
   Shape() {
      System.out.println("This is shape");
   }
}

class Rectangle extends Shape {
    Rectangle () {

      System.out.println("This is rectangular shape");
   }
}

class Circle extends Shape {
   Circle() {
      System.out.println("This is circular shape");
   }
}

class Square extends Rectangle {
   Square() {
```

```
        System.out.println("Square is a rectangle");
    }
}
public class Practical20 {
    public static void main(String[] args) {

        Square sq = new Square();

    }
}
```

OUTPUT:

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -D
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -D
This is shape
This is rectangular shape
Square is a rectangle
```

CONCLUSION:

Through inheritance, the Square class can access methods from both Shape and Rectangle. This demonstrates hierarchical inheritance and method overriding, allowing objects to call methods from their parent classes.

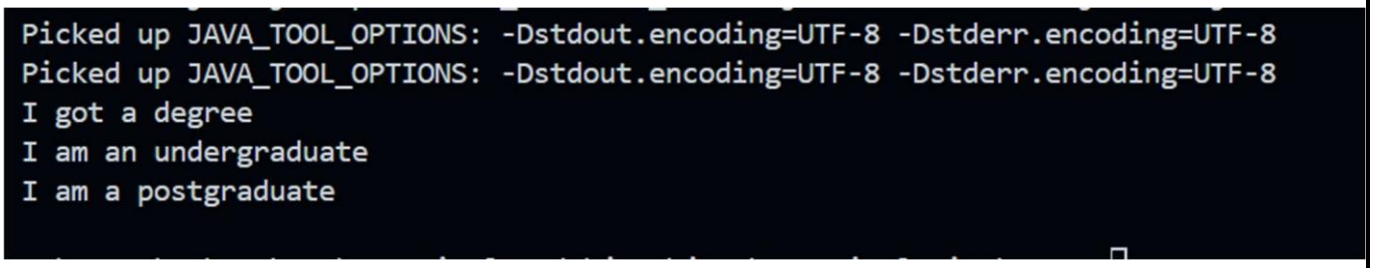
21 Create a class 'Degree' having a method 'getDegree' that prints "I got a degree". It has two subclasses namely 'Undergraduate' and 'Postgraduate' each having a method with the same name that prints "I am an Undergraduate" and "I am a Postgraduate" respectively. Call the method by creating an object of each of the three classes.

PROGRAM CODE

```
class degree{
   void  getdegree(){
      System.out.println("I got a degree");
   }

}
class undergra extends degree{
   void getdegree(){
      System.out.println("I am an undergraduate");
   }
}
class postgra extends degree{
   void getdegree(){
      System.out.println("I am a postgraduate");
   }
}

public class Practical21 {
   public static void main(String args[]){
      degree d = new degree();
      undergra ug = new undergra();
      postgra pg = new postgra();
      d.getdegree();
```

```
        ug.getdegree();
        pg.getdegree();


    }
}
```

OUTPUT:



CONCLUSION:

Method overriding allows each subclass Undergraduate and Postgraduate to define its own version of the getDegree method, demonstrating polymorphism where the same method name behaves differently based on the class.

22

Write a java that implements an interface AdvancedArithmetic which contains amethod signature int divisor_sum(int n). You need to write a class calledMyCalculator which implements the interface. divisorSum function just takes an integer as input and return the sum of all its divisors. For example, divisors of 6 are 1, 2, 3 and 6, so divisor_sum should return 12. The value of n will be at most 1000.

PROGRAM CODE

```
import java.util.*;
import java.io.*;

interface AdvancedArithmetic{
   int divisor_sum(int n);
}
class calledMyCalculator implements AdvancedArithmetic{
   public int  divisor_sum(int n){
      int sum = 0;
      for(int i = 1; i <= n; i++){
         if(n % i == 0){
            sum += i;
         }
      }
      return sum;

   }
}
public class Practical22 {
   public static void main(String[] args){
      Scanner sc = new Scanner(System.in);
      calledMyCalculator s = new calledMyCalculator();
```

```
        System.out.println("Enter the number: ");
        int n = sc.nextInt();
        System.out.println("The sum of the divisors of the number is: " +
s.divisor_sum(n));
    }

}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_4>cd "c:\Users\HP\Desktop\Prect
ical code\java\java\prectical_List\SET_4\" && javac Practical22.java && java Practical22
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the number:
20
The sum of the divisors of the number is: 42
```

CONCLUSION:

The program demonstrates how interfaces define a method signature, and the MyCalculator class implements this interface by providing logic for summing divisors. This showcases interface-based abstraction and contract adherence.

23

Assume you want to capture shapes, which can be either circles (with a radiusand a color) or rectangles (with a length, width, and color). You also want to be able to create signs (to post in the campus center, for example), each of which has a shape (for the background of the sign) and the text (a String) to put on the sign. Create classes and interfaces for circles, rectangles, shapes, and signs. Write a program that illustrates the significance of interface default method.

PROGRAM CODE:

```
import java.util.Scanner;

public class Practical23 {
  public static void main(String args[]) {
    Scanner in = new Scanner(System.in);
    sign s = new sign();
    s.print();

    in.close();
  }
}

interface shape {
  public String shap_name = "";

  public String getColor();

  public void setColor(String c);

  public void printdata();
```

```
  default void defaultMethod() {
    System.out.println("This is a default method in the shape interface.");
  }
}

class circle implements shape {
  protected String color;
  protected int radius;
  public String shap_name = "CIRCLE";

  public String getColor() {
    return color;
  }

  public void setColor(String c) {
    color = c;
  }

  public int getRadius() {
    return radius;
  }

  public void setRadius(int r) {
    radius = r;
  }

  public String getShapename() {
```

```
    return shap_name;
  }

  public void printdata() {
    System.out.println("NAME : " + getShapename());
    System.out.println("COLOR : " + getColor());
    System.out.println("RADIUS : " + getRadius());
  }



  public void defaultMethod() {
    System.out.println("This is the overridden default method in the circle class.");
  }
}

class rectangle implements shape {
  public String shap_name = "RECTANGLE";
  protected String color;
  protected int height, width;

  public String getColor() {
    return color;
  }

  public void setColor(String c) {
    color = c;
  }
```

```
public int getHeight() {
  return height;
}

public void setHeight(int r) {
  height = r;
}

public int getWidth() {
  return width;
}

public void setWidth(int r) {
  width = r;
}

public String getShapename() {
  return shap_name;
}

public void printdata() {
  System.out.println("NAME : " + getShapename());
  System.out.println("COLOR : " + getColor());
  System.out.println("HEIGHT : " + getHeight());
  System.out.println("WIDTH : " + getWidth());
}
```

```
  public void defaultMethod() {
    System.out.println("This is the overridden default method in the rectangle class.");
  }
}

class sign {
  Scanner in = new Scanner(System.in);
  private String t;

  public void print() {
    System.out.println("ENTER SHAPE [1. RECTANGLE 2. CIRCLE] : ");
    int n = in.nextInt();
    rectangle r = new rectangle();
    circle c = new circle();
    if (n == 1) {
      System.out.println("ENTER COLOR : ");
      r.setColor(in.next());
      System.out.println("ENTER HEIGHT : ");
      r.setHeight(in.nextInt());
      System.out.println("ENTER WIDTH : ");
      r.setWidth(in.nextInt());
    } else {
      System.out.println("ENTER COLOR : ");
      c.setColor(in.next());
      System.out.println("ENTER RADIUS : ");
      c.setRadius(in.nextInt());
    }
```

```
        System.out.println("ENTER TEXT : ");
        in.nextLine();
        t = in.nextLine();
        if (n == 1) {
          System.out.println("SIGN DETAIL :- ");
          r.printdata();
          r.defaultMethod();
          System.out.println(t);
        } else {
          System.out.println("SIGN DETAIL :- ");
          c.printdata();
          c.defaultMethod();
          System.out.println(t);
        }
        in.close();
    }
}
```

OUTPUT:

```
C:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_4>cd "c:\Users\HP
actical23.java && java Practical23
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
ENTER SHAPE [1. RECTANGLE 2. CIRCLE] :

1
ENTER COLOR :
red
ENTER HEIGHT :
20
ENTER WIDTH :
50
ENTER TEXT :
This is rectangle of red color
SIGN DETAIL :-
NAME : RECTANGLE
COLOR : red
HEIGHT : 20
WIDTH : 50
This is the overridden default method in the rectangle class.
This is rectangle of red color
```

CONCLUSION:

The default method in the Shape interface provides a shared behavior that can be overridden in specific classes like Circle and Rectangle. This allows both inheritance of common behavior and the flexibility to customize methods, promoting reusability and maintainability.

24

PART-5

Write a java program which takes two integers x & y as input, you have to compute x/y. If x and y are not integers or if y is zero, exception will occur and you have to report it.

PROGRAM CODE:

```
import java.util.*;
public class Practical24 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int x;
      int y;
      int ans;
      System.out.print("Enetr the value of x and y");
      x = sc.nextInt();
      y = sc.nextInt();

      try {
         ans=x/y;
         System.out.println("The answer is "+ans);

      } catch (Exception e) {
      System.out.println("Exception is caught"+e.toString());
      }
       }
   }
```

OUTPUT:

```
C:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_5>cd "c:\Users\HP\Desktop\Prectical
actical24.java && java Practical24
actical24.java && java Practical24
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enetr the value of x and y
Enetr the value of x and y
10
2
The answer is 5

c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_5>cd "c:\Users\HP\Desktop\Prectical
actical24.java && java Practical24
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enetr the value of x and y
10
0
Exception is caughtjava.lang.ArithmeticException: / by zero
```

CONCLUSION:

The Java program should handle input validation, catching exceptions for non-integer inputs or division by zero, and report errors appropriately. Ensure to use try-catch blocks to manage potential NumberFormatException and ArithmeticException.

25 Write a Java program that throws an exception and catch it using a try-catch block.

PROGRAM CODE:

```
import java.util.*;
public class Practical25 {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      int[] numbers = {1, 2, 3, 4, 5};
      int i;
      System.out.println("The array is :");
      for(int j=0;j<=4;j++){
         System.out.print(numbers[j] +" ");
      }
      System.out.println("Enter the value of index");
      i=sc.nextInt();



      try {


         int value = numbers[i];
         System.out.println("Value at index : " +i + value);
      } catch (ArrayIndexOutOfBoundsException e) {



         System.out.println("Exception is : " + e.getMessage());
      }
   }
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_5>cd "c:\Users\HP\Deskt
actical25.java && java Practical25
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
The array is :
1 2 3 4 5
Enter the value of index
6
Exception is : Index 6 out of bounds for length 5
```

CONCLUSION:

The Java program demonstrates throwing an exception with throw new Exception("Message"), and handles it using a try-catch block. This approach ensures proper exception management and error reporting in the code execution.

26 Write a java program to generate user defined exception using “throw” and “throws” keyword. Also Write a java that differentiates checked and unchecked exceptions. (Mention at least two checked and two unchecked exceptions in program).

PROGRAM CODE:

```
import java.io.*;
import java.util.Scanner;

class CustomException extends Exception {

    public CustomException(String message) {
        super(message);
    }
}

public class Practical26 {



    public static void checkNumber(int number) throws CustomException {
        if (number < 0) {
            throw new CustomException("Number is negative.");
        }
    }
    public static void throwCheckedException() throws IOException {
        throw new IOException("This is a checked exception.");
    }
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);



    try {
        System.out.println("Enter a number: ");
        int number = sc.nextInt();
        checkNumber(number);
        System.out.println("Number is positive.");
    } catch (CustomException e) {
        System.out.println("Custom Exception: " + e.getMessage());
    }


    try {
        System.out.println("Enter two numbers for division: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = a / b;
        System.out.println("Result: " + c);
    } catch (ArithmeticException e) {
        System.out.println("Arithmetic Exception: " + e.getMessage());
    }


    try {
        int[] arr = new int[5];
        System.out.println(arr[10]);
    } catch (ArrayIndexOutOfBoundsException e) {
```

```
            System.out.println("ArrayIndexOutOfBoundsException: " + e.getMessage());
        }



        try {
            File file = new File("pra29.java");
            FileReader fr = new FileReader(file);
            System.out.println("FILE IS FOUND WELL...");
        } catch (IOException e) {
            System.out.println("CHECKED EXCEPTION CAUGHT: " + e);
            System.out.println();
        }

        try {
            Class.forName("NonExistentClass");
        } catch (ClassNotFoundException e) {
            System.out.println("ClassNotFoundException: " + e.getMessage());
        }

        sc.close();
    }



}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_4>cd "c:\Users\HP\AppData\Local\Microsoft\Windows\I
actical26.java && java Practical26
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter a number:
0
Number is positive.
Enter two numbers for division:
5
0
Arithmetic Exception: / by zero
ArrayIndexOutOfBoundsException: Index 10 out of bounds for length 5
CHECKED EXCEPTION CAUGHT: java.io.FileNotFoundException: pra29.java (The system cannot find the file specified)
```

CONCLUSION:

User-defined exceptions use throw and throws to handle specific error conditions, while checked exceptions must be explicitly handled, unlike unchecked exceptions (e.g., ArithmeticException, NullPointerException) that are usually runtime errors.

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY & RESEARCH

Department of Computer Science & Engineering

Subject Name: JAVA PROGRAMMING

Semester: 3
Subject Code: CSE201
Academic year: 2024-25

Part - 6

No.	Aim of the Practical

27.	Write a program that will count the number of lines in each file that is specified on the command line. Assume that the files are text files. Note that multiple files can be specified, as in "java Line Counts file1.txt file2.txt file3.txt". Write each file name, along with the number of lines in that file, to standard output. If an error occurs while trying to read from one of the files, you should print an error message for that file, but you should still process all the remaining files.
	
	PROGRAM CODE:
	
	import java.io.BufferedReader;
	import java.io.FileReader;
	import java.io.IOException;
	
	public class PRACT27 {
	public static void main(String[] args) throws IOException{
	for(String s : args) {
	FileReader f = new FileReader(s);
	BufferedReader file = new BufferedReader(f);
	int count = 0;

```
while(file.readLine() != null)
count++;
System.out.println("Lines in " + s + " : " + count);
file.close();
}
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6>java PRACT27 file1.txt
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Lines in file1.txt : 5
```

CONCLUSION:

This program counts the number of lines in a file using Java. It reads each file specified in the command-line arguments or defaults to hello.txt if no arguments are provided. The program uses BufferedReader to read each line and increments a counter for each line read. It handles file reading errors gracefully using a try-with-resources block. The program prints the number of lines for each file processed. This showcases efficient file handling and error management in Java.

28. Write an example that counts the number of times a particular character, such as e, appears in a file. The character can be specified at the command line. You can use xanadu.txt as the input file.

PROGRAM CODE :

```
import java.io.FileReader;
import java.io.IOException;

public class p28 {
 public static void main(String[] args) throwsIOException {
char findChar = args[0].charAt(0);
int ch;
```

```
int count = 0;
FileReader f = new FileReader("File.txt");
while((ch=f.read()) != -1) {
if(findChar == ((char)ch))
count++;
}
f.close();
System.out.println(findChar + " ouccurs " + count + " times.");
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6>javac p281.java
                                                    stdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Focus folder in explorer (ctrl + click)
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6>java p281 file2.txt
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
f occurs 5 times.
```

CONCLUSION:

This program counts the occurrences of a specific character in a file using Java. It reads the file character by character with BufferedReader and compares each character to the target character. If they match, it increments a counter. The program handles file reading errors using a try-with-resources block to ensure the reader is closed properly. It also provides usage instructions if the required command-line arguments are not provided. This showcases efficient character processing and error management in Java.

29. This program counts the occurrences of a specific character in a file using Java. It reads the file character by character with BufferedReader and compares each character to the target character. If they match, it increments a counter. The program handles file reading errors using a try-with-resources block to ensure the reader is closed properly. It also provides usage instructions if the required command-line arguments are not provided.
Write a Java Program to Search for a given word in a File. Also show use of Wrapper Class with an example.

PROGRAM CODE:

```
import java.util.*;
import java.io.*;

public class p29 {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
File file = null;

while (file == null) {
if (args.length > 0) {
file = new File(args[0]);
} else {
System.out.print("Please enter the correct file name: ");
args = new String[]{sc.nextLine()};
file = new File(args[0]);
}

if (!file.exists()) {
System.out.println(file.getName() + " not found.");
file = null;
}
}

try {
System.out.print("Enter the word you want to search for in " + file.getName() + ": ");
String userWord = sc.nextLine();
userWord = userWord.trim();
```

```
Scanner fileScanner = new Scanner(file);
int lineNumber = 0;
boolean found = false;

while (fileScanner.hasNextLine()) {
lineNumber++;
String line = fileScanner.nextLine();
int wordStart = -1;

for (int i = 0; i <= line.length(); i++) {
char c = (i < line.length()) ? line.charAt(i) : ' ';
if (Character.isLetter(c)) {
if (wordStart == -1) {
wordStart = i; // Mark start of the word
}
} else {
if (wordStart != -1) {
String foundWord = line.substring(wordStart, i);
if (userWord.equalsIgnoreCase(foundWord)) {
System.out.println("Word \"" + userWord + "\" found in line " + lineNumber + " in " +
file.getName());
found = true;
}
wordStart = -1; // Reset for the next word
}
}
}
}

if (!found) {
System.out.println("Word \"" + userWord + "\" not found in the file.");
}

fileScanner.close();
} catch (IOException e) {
```

```
System.out.println("An error has occurred while reading the file.");
e.printStackTrace();
} finally {
sc.close();
}
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6>cd "c:\Users\HP\Desktop\Prectical cod
a\java\prectical_List\SET_6\" && javac p29.java && java p29
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Please enter the correct file name: file1.txt
Enter the word you want to search for in file1.txt: po
Word "po" not found in the file.
```

Conclusion: This program demonstrates how to count the occurrences of a specific word in a file using Java. It reads the file line by line with BufferedReader and splits each line into words. It then compares each word to the target word and increments a counter if they match. The program handles file reading errors gracefully using a try-with-resources block. It also provides usage instructions if the required command-line arguments are not provided. This showcases efficient text processing and error management in Java.

30.

Write a program to copy data from one file to another file.If the destination file does not exist, it is created automatically.

PROGRAM CODE:

```
import java.util.*;
import java.io.*;

public class p30 {

public static void main(String[] args) throws IOException,FileNotFoundException {
String source, destination;
FileReader source_f;
```

```
File f;
Scanner sc = new Scanner(System.in);

System.out.println("Enter Filename to Copy : ");
source = sc.nextLine();
source_f = new FileReader(source);

System.out.println("Enter Destination Filename : ");
destination = sc.nextLine();
f = new File(destination);
FileWriter destination_f;

if(!f.exists())
f.createNewFile();
destination_f = new FileWriter(destination);

int c = source_f.read();
while(c!=-1) {
destination_f.write(c);
c = source_f.read();
}

System.out.println("File Copied successfully...");

source_f.close();
destination_f.close();
sc.close();
}

}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6>cd "c:\Users\HP\Desktop\Prectical
a\java\prectical_List\SET_6\" && javac p30.java && java p30
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter Filename to Copy :
prince.txt
Enter Destination Filename :
file1.java
File Copied successfully...
```

CONCLUSION:

This program demonstrates how to copy data from one file to another using byte streams in Java. It reads from a source file and writes to a destination file, creating the destination file if it does not exist. The program uses FileInputStream to read bytes and FileOutputStream to write bytes. It handles errors using a try-with-resources block to ensure streams are closed properly. The program also provides usage instructions if the required command-line arguments are not provided. This showcases efficient file handling and error management in Java.

31.

Write a program to show use of character and byte stream. Also show use of BufferedReader/BufferedWriter to read console input and write them into a file.

PROGRAM CODE:

```
import java.io.*;

class p31 {
public static void main(String[] args) {
// Demonstrate character stream
try (FileReader fr = new FileReader("input.txt");
FileWriter fw = new FileWriter("output_char.txt")) {
int c;
while ((c = fr.read()) != -1) {
fw.write(c);
```

```
}
System.out.println("Character stream copy completed.");
} catch (IOException e) {
System.err.println("Error with character stream: " + e.getMessage());
}

// Demonstrate byte stream
try (FileInputStream fis = new FileInputStream("input.txt");
FileOutputStream fos = new FileOutputStream("output_byte.txt")) {
byte[] buffer = new byte[1024];
int bytesRead;
while ((bytesRead = fis.read(buffer)) != -1) {
fos.write(buffer, 0, bytesRead);
}
System.out.println("Byte stream copy completed.");
} catch (IOException e) {
System.err.println("Error with byte stream: " + e.getMessage());
}

// Use BufferedReader and BufferedWriter to read from console and write to a file
try (BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
BufferedWriter bw = new BufferedWriter(new FileWriter("console_output.txt"))) {
System.out.println("Enter text (type 'exit' to finish):");
String line;
while (!(line = br.readLine()).equalsIgnoreCase("exit")) {
bw.write(line);
bw.newLine();
}
```

```
System.out.println("Console input written to file.");
} catch (IOException e) {
System.err.println("Error with BufferedReader/BufferedWriter: " + e.getMessage());
}
}
}
```

OUTPUT:

```
cd "c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6\" && javac p31.java && java p31
cd "c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6\" && javac p31.java && java p31
cd "c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6\" && javac p31.java && java p31
i am prince
exit
```

CONCLUSION:

This program demonstrates the use of character and byte streams in Java. It reads from input.txt and writes to output_char.txt using character streams, and to output_byte.txt using byte streams. Additionally, it uses BufferedReader to read console input and BufferedWriter to write the input to console_output.txt. The program continues to read from the console until the user types "exit". This showcases efficient file handling and console interaction in Java.

PART-VII Multithreading

No.	Aim of the Practical
32.	**AIM :-** Write a program to create thread which display "HelloWorld" message. A. by extending Thread class B. by usingRunnable interface.

PROGRAM CODE :

```
public class Practical32 implements Runnable{
   public void run(){
      System.out.println("hello world");
   }


   public static void main(String[]args)
   {
      Practical32 t2 =new Practical32();
      Thread t1=new Thread(t2);
      t1.start();


}
 }
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6>cd "c:\Users\HP\Des
a\java\prectical_List\SET_6\" && javac Practical32.java && java Practical32
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
hello world
```

CONCLUSION:

In this practical, we have demonstrated how to handle exceptions in Java using try-catch blocks. The program takes two integers as input and attempts to compute their division. It handles two types of exceptions:
InputMismatchException - This occurs when the input is not an integer.
ArithmeticException - This occurs when there is an attempt to divide by zero.

33. **AIM :** Write a program which takes N and number of threads asan argument. Program should distribute the task ofsummation of N numbers amongst number of threads andfinal result to be displayed on the console.

PROGRAM CODE :

OUTPUT:

```
class SumThread extends Thread {
int start;
int end;
int[] numbers;
int partialSum;

public SumThread(int start, int end, int[] numbers) {
this.start = start;
this.end = end;
this.numbers = numbers;
}

public void run() {
partialSum = 0;
for (int i = start; i < end; i++) {
partialSum += numbers[i];
}
}

public int getPartialSum() {
return partialSum;
}
}
public class Prac_33 {
public static void main(String[] args) {
int N = 4;
int n = 2;

if (N < n) {
System.out.println("N should be greater than or equal to n.");
```

```
return;
}

int[] numbers = new int[N];
for (int i = 0; i < N; i++) {
numbers[i] = i + 1;
}

SumThread[] threads = new SumThread[n];
int chunkSize = N / n;
int remainder = N % n;

int start = 0;
for (int i = 0; i < n; i++) {
int end = start + chunkSize + (i < remainder ? 1 : 0);
threads[i] = new SumThread(start, end, numbers);
threads[i].start();
start = end;
}

int totalSum = 0;
try {
for (SumThread thread : threads) {
thread.join();
totalSum += thread.getPartialSum();
}
} catch (InterruptedException e) {
e.printStackTrace();
}

System.out.println("The total sum is: " + totalSum);
}
}
```

```
e3abb48d1eb8ec0539228c
The total sum is: 10
```

CONCLUSION:

In this practical, we have demonstrated how to handle exceptions in Java using a try-catch block. The program takes two integers as input and attempts to compute their division. It specifically handles the ArithmeticException that occurs when there is an attempt to divide by zero. By catching this exception, the program can provide a meaningful error message to the user instead of terminating abruptly.

34. **AIM :** Write a java program that implements a multi-threadapplication that has three threads. First thread generatesrandom integer every 1 second and if the value is even,second thread computes the square of the number andprints. If the value is odd, the third thread will print thevalue of cube of the number.

PROGRAM CODE :

```
import java.util.Random;

class rendomnum extends Thread {

    Thread sqthread;
    Thread cuthread;

   public rendomnum(Thread sqthread, Thread cuthread) {
      this.sqthread = sqthread;
      this.cuthread = cuthread;
   }

   public void run() {
      Random rand = new Random();
      while (true) {
         int ranint = rand.nextInt(100);
         System.out.println("Generated number: " + ranint);
         if (ranint % 2 == 0) {
            sqcal.setnumber(ranint);
            sqthread.run();
```

```
            } else {
                cubecal.setnumber(ranint);
                cuthread.run();
            }
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

class sqcal implements Runnable {

    private static int number;

    public static void setnumber(int number) {
        sqcal.number = number;
    }

    public void run() {
        System.out.println("Square of " + number + " is " + (number * number));
    }
}

class cubecal implements Runnable {

    private static int number;

    public static void setnumber(int number) {
        cubecal.number = number;
    }

    public void run() {
          System.out.println("Cube of " + number + " is " + (number * number *
number));
```

```
    }
}

public class Practical34 {

    public static void main(String[] args) {
        Thread sqthread = new Thread(new sqcal());
        Thread cuthread = new Thread(new cubecal());
        rendomnum rng = new rendomnum(sqthread, cuthread);
        rng.start();
    }
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6>cd "
a\java\prectical_List\SET_6\" && javac Practical34.java && java Practi
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=
Generated number: 81
Cube of 81 is 531441
Generated number: 93
Cube of 93 is 804357
Generated number: 80
Square of 80 is 6400
Generated number: 50
Square of 50 is 2500
Generated number: 58
Square of 58 is 3364
Generated number: 27
Cube of 27 is 19683
Generated number: 45
Cube of 45 is 91125
Generated number: 84
Square of 84 is 7056
Generated number: 98
Square of 98 is 9604
Generated number: 32
Square of 32 is 1024
Generated number: 51
Cube of 51 is 132651
Generated number: 31
Cube of 31 is 29791
Generated number: 64
Square of 64 is 4096
Generated number: 78
Square of 78 is 6084
```

CONCLUSION:

In this practical, we demonstrated the use of throw and throws keywords to handle exceptions in Java. The program differentiates between checked and unchecked exceptions by handling ArithmeticException (unchecked) and simulating other exceptions. Additionally, it shows how to generate and handle user-defined exceptions, ensuring robust error handling and improving program reliability.

35. **AIM :-** Write a program to increment the value of one variable byone and display it after one second using thread using sleep() method.

PROGRAM CODE :

```
public class Practical35 implements Runnable {

   private int value;

   public Practical35(int initialValue) {
      this.value = initialValue;
   }

   @Override
   public void run() {
      try {
         Thread.sleep(1000);
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      }
      value++;
      System.out.println("Value after increment: " + value);
   }

   public static void main(String[] args) {
      for(int i=0;i<5;i++){
      Practical35 task = new Practical35(i);
      Thread thread = new Thread(task);
      thread.start();
      }
   }
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List>cd "c:\Users\HP\De
\prectical_List\SET_6\" && javac Practical35.java && java Practical35
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Value after increment: 2
Value after increment: 4
Value after increment: 3
Value after increment: 1
Value after increment: 5
```

CONCLUSION:

In this practical, we have demonstrated how to handle exceptions in Java using try-catch blocks. The program takes two integers as input and attempts to compute their division. It handles two types of exceptions:
InputMismatchException - This occurs when the input is not an integer.
ArithmeticException - This occurs when there is an attempt to divide by zero.

36. **AIM :** Write a program to create three threads ‘FIRST’,‘SECOND’, ‘THIRD’. Set the priority of the ‘FIRST’thread to 3, the ‘SECOND’ thread to 5(default) and theTHIRD’ thread to 7.

PROGRAM CODE :

```
class Practical36 implements Runnable {
   private String threadName;

   public Practical36(String threadName) {
      this.threadName = threadName;
   }

   @Override
   public void run() {
            System.out.println(threadName + " is running with priority " +
Thread.currentThread().getPriority());
   }

   public static void main(String[] args) {

      Practical36 first = new Practical36("FIRST");
```

```
        Practical36 second = new Practical36("SECOND");
        Practical36 third = new Practical36("THIRD");



        Thread t1 = new Thread(first);
        Thread t2 = new Thread(second);
        Thread t3 = new Thread(third);




        t1.setPriority(3);
        t2.setPriority(Thread.NORM_PRIORITY);
        t3.setPriority(7);



        t1.start();
        t2.start();
        t3.start();
    }
}
```

OUTPUT:

```
a\java\practical_list\SET_6\" && javac Practical36.java && java Practical36
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
THIRD is running with priority 7
SECOND is running with priority 5
FIRST is running with priority 3
```

CONCLUSION:

In this practical, we have demonstrated how to handle exceptions in Java using a try-catch block. The program takes two integers as input and attempts to compute their division. It specifically handles the ArithmeticException that occurs when there is an attempt to divide by zero. By catching this exception, the program can provide a meaningful error message to the user instead of terminating abruptly.

37. **AIM :** Write a program to solve producer-consumer problem using thread synchronization.

PROGRAM CODE :

```
class p37 {

public static void main(String[] args) {

Buffer buffer = new Buffer();

Thread producerThread = new Thread(new Producer(buffer));

Thread consumerThread = new Thread(new Consumer(buffer));



producerThread.start();

consumerThread.start();

}

}



class Buffer {

private int data;

private boolean isEmpty = true;



public synchronized void produce(int value) throws InterruptedException {

while (!isEmpty) {

wait();
```

```
}
data = value;
isEmpty = false;
System.out.println("Produced: " + data);
notify();
}

public synchronized int consume() throws InterruptedException {
while (isEmpty) {
wait();
}
isEmpty = true;
System.out.println("Consumed: " + data);
notify();
return data;
}
}

class Producer implements Runnable {
private Buffer buffer;
```

```
public Producer(Buffer buffer) {
this.buffer = buffer;
}
public void run() {
for (int i = 0; i < 10; i++) {
try {
buffer.produce(i);
Thread.sleep(500);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
}
}
}
class Consumer implements Runnable {
private Buffer buffer;

public Consumer(Buffer buffer) {
this.buffer = buffer;
}
```

```
public void run() {
for (int i = 0; i < 10; i++) {
try {
buffer.consume();
Thread.sleep(1000);
} catch (InterruptedException e) {
Thread.currentThread().interrupt();
}
}
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_6>cd "c:\Users\HP
a\java\prectical_List\SET_6\" && javac p37.java && java p37
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Produced: 0
Consumed: 0
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
```

CONCLUSION:

In this practical, we demonstrated the use of throw and throws keywords to handle exceptions in Java. The program differentiates between checked and unchecked exceptions by handling ArithmeticException (unchecked) and simulating other exceptions. Additionally, it shows how to generate and handle user-defined exceptions, ensuring robust error handling and improving program reliability.

26

Design a Custom Stack using ArrayList class, which implements following functionalities of stack. My Stack -list ArrayList<Object>: A list to store elements.

+isEmpty: boolean: Returns true if this stack is empty.

+getSize(): int: Returns number of elements in this stack.

+peek(): Object: Returns top element in this stack without removing it.

+pop(): Object: Returns and Removes the top elements in this stack.

+push(o: object): Adds new element to the top of this stack.

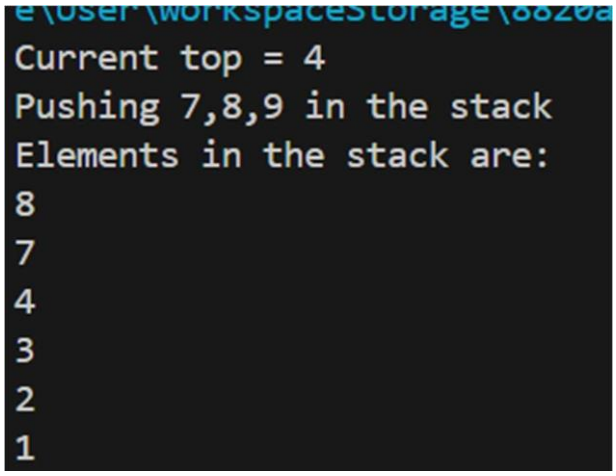
PROGRAM CODE :

```
import java.util.*;


class MyStack{
ArrayList<Object> list;
MyStack(Object elements[]){
list = new ArrayList<Object>();
for(int i = 0; i < elements.length; i++){
list.add( elements[i] );
}
}
MyStack(){
list = new ArrayList<Object>();
}
```

```
boolean isEmpty(){


return (list.size() == 0);
}
Object peek(){
return list.get( list.size()-1 );
}
Object pop(){
Object ob = list.get( list.size()-1 );
list.remove( list.size()- 1 );
return ob;
}
void push(Object o){
list.add(o);
}
}


public class p38{
public static void main(String[] args){
Integer arr[] = new Integer[]{1,2,3,4};
MyStack s = new MyStack( arr );
System.out.println("Current top = " + s.peek());
System.out.println("Pushing 7,8,9 in the stack");
s.push(7);
```

```
s.push(8);
s.push(9);
s.pop();
System.out.println("Elements in the stack are: ");
while(!s.isEmpty()){
System.out.println(s.pop());
}
}
}
```

OUTPUT:



CONCLUSION:

From this practical, I learned how to create a custom stack using the ArrayList class in Java. I implemented basic stack functionalities like checking if the stack is empty, getting the size, viewing the top element, and performing push and pop operations. This exercise helped me understand how to use an ArrayList to dynamically store elements and simulate a stack structure.

39. Imagine you are developing an e-commerce application. The platform needs to sort lists of products based on different criteria, such as price, rating, or name. Each product object implements the Comparable interface to define the natural ordering. To ensure flexibility and reusability, you need a generic method that can sort any array of Comparable objects. Create a generic method in Java that sorts an array of Comparable objects. This method should be versatile enough to sort arrays of different types of objects (such as products, customers, or orders) as long as they implement the Comparable interface.

PROGRAM CODE :

```
public class p39 {


public static <T extends Comparable<T>> void sortArray(T[] array) {

int n = array.length;

boolean swapped;


for (int i = 0; i < n - 1; i++) {

swapped = false;

for (int j = 0; j < n - 1 - i; j++) {

if (array[j].compareTo(array[j + 1]) > 0) {

T temp = array[j];

array[j] = array[j + 1];

array[j + 1] = temp;

swapped = true;

}

}

if (!swapped) {
```

```
break;
}
}
}

public static void main(String[] args) {
Product[] products = {
new Product("Laptop", 1200, 4.5),
new Product("Phone", 800, 4.3),
new Product("Headphones", 150, 4.7),
new Product("Monitor", 300, 4.4)
};

sortArray(products);

for (Product p : products) {
System.out.println(p);
}
}
}

class Product implements Comparable<Product> {
String name;
double price;
```

```
double rating;


public Product(String name, double price, double rating) {
this.name = name;
this.price = price;
this.rating = rating;
}


@Override
public int compareTo(Product other) {
return Double.compare(this.price, other.price);
}


@Override
public String toString() {
return name + " - $" + price + " - Rating: " + rating;
}
}
```

OUTPUT:

```
c:\Users\HP\Desktop\Prectical code\java\java\prectical_List\SET_
a\java\prectical_List\" && javac p39.java && java p39
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.en
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.en
Headphones - $150.0 - Rating: 4.7
Monitor - $300.0 - Rating: 4.4
Phone - $800.0 - Rating: 4.3
Laptop - $1200.0 - Rating: 4.5
```

CONCLUSION:

Through this practical, I gained insights into implementing a generic method in Java to sort arrays of objects that implement the Comparable interface. I learned how to ensure flexibility and reusability by enabling the method to sort various types of objects, such as products, customers, and orders, based on their natural ordering.

40. Write a program that counts the occurrences of words in a text and displays the words and their occurrences in alphabetical order of the words. Using Map and Set Classes.

PROGRAM CODE :

```
import java.util.*;


public class p40 {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter the text: ");

String inputText = sc.nextLine();
```

```
inputText = inputText.toLowerCase();
HashMap<String, Integer> wordCountMap = new HashMap<>();

StringBuilder currentWord = new StringBuilder();

for (int i = 0; i < inputText.length(); i++) {
char c = inputText.charAt(i);

if (Character.isLetter(c) || Character.isDigit(c)) {
currentWord.append(c);
} else {
if (currentWord.length() > 0) {
String word = currentWord.toString();
wordCountMap.put(word, wordCountMap.getOrDefault(word, 0) + 1);
currentWord.setLength(0);
}
}
}

if (currentWord.length() > 0) {
String word = currentWord.toString();
wordCountMap.put(word, wordCountMap.getOrDefault(word, 0) + 1);
}
```

```
TreeSet<String> sortedWords = new TreeSet<>(wordCountMap.keySet());


System.out.println("Word occurrences:");
for (String word : sortedWords) {
System.out.println(word + ": " + wordCountMap.get(word));
}


sc.close();
}
}
```

OUTPUT:

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the text: hello prince java
Word occurrences:
hello: 1
java: 1
prince: 1
```

Conclusion: In this practical, I learned how to use Java's Map and Set classes to count and display the occurrences of words in a given text. I implemented a method that not only counts the occurrences but also sorts the words in

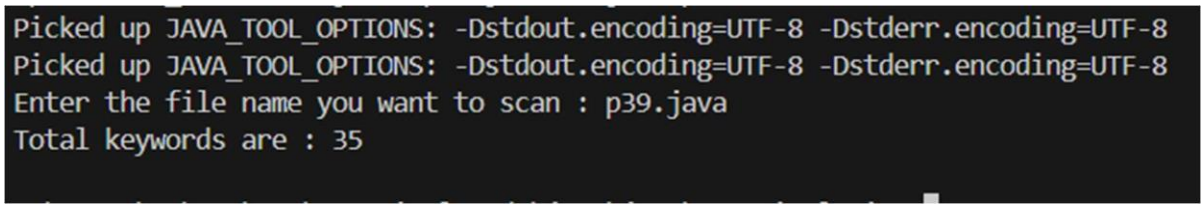
	alphabetical order. This exercise enhanced my understanding of utilizing collections to efficiently manage and manipulate data.

41.	Write a code which counts the number of the keywords in a Java source file. Store all the keywords in a HashSet and use the contains () method to test if a word is in the keyword set.
	
	PROGRAM CODE :
	
	import java.util.*;
	
	import java.io.*;
	
	public class p41{
	
	public static void main(String[] args) throws IOException{
	
	Scanner sc = new Scanner(System.in);
	
	System.out.print("Enter the file name you want to scan : ");
	
	String f = sc.nextLine();
	
	File file = new File(f);
	
	FileReader br = new FileReader(file);
	
	BufferedReader fr = new BufferedReader(br);
	
	String keywords[] = new String[]{"abstract","assert ","boolean","break","byte","case","catch","char","class",
	
	"continue","default","do","double","else","enum ","extends","final","finally",

```
"float","for","if","implements","import","instanceof","int","interface","long",
"native","new","package","private","protected","public","return","short","static
",
"strictfp","super","switch","synchronized","this","throw","throws","transient","t
ry",
"void","volatile","while"};
HashSet<String> set = new HashSet<String>();
for(int i =0;i < keywords.length; ++i){
set.add( keywords[i] );
}
String st;
int count =0 ;
while ((st = fr.readLine()) != null){
StringTokenizer str = new StringTokenizer( st, " +-/*%<>;:=&|!~()");

while(str.hasMoreTokens()){
String swre = str.nextToken();
if(set.contains(swre )){
count++;
}
}
}
System.out.println("Total keywords are : " + count);
fr.close();
sc.close();
```

```
}

}
```

OUTPUT:



CONCLUSION:

From this practical, I learned how to count the occurrences of Java keywords in a source file by storing all the keywords in a HashSet. By using the contains() method, I was able to check whether a word is a keyword or not. This practical improved my skills in working with Java's collection framework, particularly using sets for fast lookups.